Theory Questions

1. Perception

1- Perception Sensors:

Radars:

Send radio waves to detect objects around and analyze the reflected back signals, mainly measuring their speed, size, and distance away. Works in fog and rain, but it does not tell what the object is precisely.

LiDARs:

Send pulses of laser light and measure how much time it takes for the pulses to return back from the objects around, building a high resolution, very detailed 3D map for the different things around the car. Perform poorly in fog and rain nevertheless and need specialized processing to work well.

Cameras:

capture light and color around the car, producing a 2D image or video stream. Do not give direct depth information and are very sensitive to lighting conditions.

2- Stereo Vision:

Basically, two cameras at a known baseline distance see the same point at different disparity, which is the difference in pixel position of a point. If an object is close, it has more disparity, and the opposite is if it's far. That, combined with the focal lense of the camera and its known real position can give us the position of the object in 3D, it's very similar to how humans see things in 3D.

3- Machine Learning:

Neural networks:

neural networks are a model that stacks simple nodes in layers with a connection between, applying a weighted sum and nonlinear function to avoid collapsing into a linear single operation, and finally processing data. They learn by backpropagation by comparing the predicted output with the real one and measuring the difference between them, and with training that difference decreases, by adjusting the weights via gradient descent, which determines the direction and amount to change each weight to reduce the error.

How does it aid the sensor?

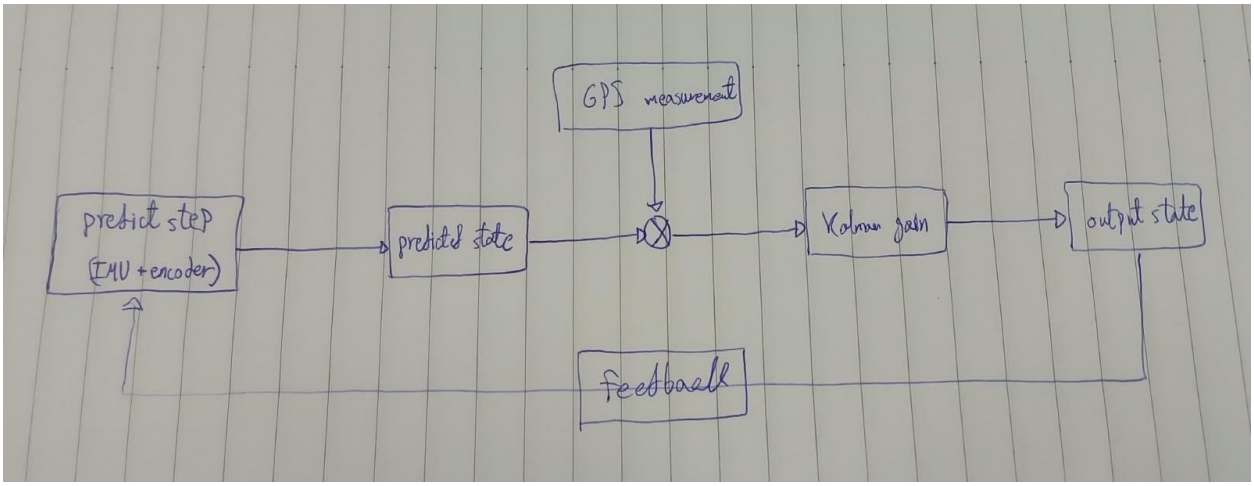
It helps with turning the numbers and photos and video streams of the sensors into actual meaning for the car to know what exactly the object is, camera just generates pixels, a LiDAR scan is just points, they alone are not useful for the car. So neural networks by learning and training on sensors and processing their data with time can convert the 3D points into real sentences like "there is a car 8 meters away", or using the camera 2D image to say that "There is a cone ahead".

2. State Estimation

1- Kalman Filter:

Kalman filter is a two-step algorithm. First, predict or estimate where you will be next, then update and correct that estimation when the real movement happens using a sensor measurement, weighted by how much you trust the prediction vs the measurement via the "Kalman gain". Then using the two information we can get more precise parameters. In the car, it can estimate and correct the position or the speed of it. It's used to reduce sensor noises because we cannot trust sensors directly, neither the predictions, The measurement here is like a GPS reading; there are cases where we can trust the reading more, and others in which we can trust the prediction more, like when being in a tunnel or a closed place, the GPS is not accurate.

BLOCK DIAGRAM



The project: we need to know primary things about the car like its position, which way it's heading, its speed, and how fast it's turning, all depending on the goal of the car. I would use GPS and IMU combined to know the car's position and turning speed, a wheel encoder to know the speed of the car itself. IMU and the encoder here are used to predict the car movement in the future, while the GPS is used to measure where the car is right now

2- SLAM:

SLAM is a technique in which we try to do the localizing and mapping simultaneously, because normally localization needs a map to check against, and to make a map we need to know where the car is first. But with SLAM we can do both at the same time. There are many types of it, like EKF-SLAM which depends on Kalman filter, FastSLAM, and Graph-based SLAM.

How can we build a map in SLAM?

First, the car needs to record where the physical objects around it exist, relative to its movement on the road. It places a new point in a growing map every time a sensor reads something around. For this task, we might need LiDAR sensors to know the shape and position of each object around precisely. We might also need cameras to know the visual features of the objects; this can be useful as a landmark without dense range data. Radar sensors might also be helpful in case of a dusty or rainy environment, but not mainly.

Vehicle localization in SLAM technique is a continuous operation as the car moves along, depending on the new sensor reading and what the car already mapped, it predicts where it is most likely to be now, using the growing map it's building at the time.

In EKF-SLAM: the prediction step tells the car position forward using motion, but in the update and correct step, it doesn't use a regular type of map like GPS; it rather uses the growing map it's building at the time to reduce the error.

In Graph-based SLAM: this approach builds a graph over time, representing each new position (of the car or the objects around) as a node on it, and representing the relative measurements between them with edges. Localization then comes after solving the graph as an optimization problem.

3. Graph Search Algorithms

1- Algorithms:

BFS: it explores the graph level by level beginning with the start node and what's around it, then what's around them and so on, but only works if the edges are equal in cost or unweighted. Simple and has the shortest path but slow and memory heavy.

DFS: explores one path deeply before tracking back and exploring another. Uses less memory than BFS but does not guarantee the shortest path.

Dijkstra's: same as BFS but works with the weighted edges, and instead of exploring one node by one, it always expands in the way of the cheapest known total cost from the start. Has the shortest weighted path but still explores blindly in all directions.

A*: same idea as Dijkstar's, but with an estimate of the remaining distance to the goal, it prioritizes exploring nodes that are likely to be heading to the goal. guarantees the shortest path if the heuristic never overestimates the actual distance and is faster than Dijkstra's. but its performance depends on having a good heuristic.

2- Vacuum Cleaning Robot:

The main problem here is the graph itself, not the algorithm. If we map the road in like grid of squares next to each other (the graph) the vacuum is limited to move right, left, forward or backward, the A* algorithm chooses the shortest path, but its movement is limited in this situation of a zig zag road. My solution here would be smoothing the path made by the algorithm after the search is done, like making it in curves and not hard turns. But the risk here is that the smoothed path neglects something the algorithm carefully avoided like an obstacle in the middle of the road making the robot hit it, in that case it would slow down the operation or even cause damage to the robot itself.

4. Control

1- Control Theory and Philosophy:

Control in engineering means to manipulate the input deliberately to reach the desired output, such as car speed or acceleration. It's how to make the system behave the way I want it considering the sensor noises and the environment disturbances. Most engineering problems are built on the control theory, like if I want a certain room temperature, or maintain the speed of a car. It's a mathematical way to reach something rather than depending on a guess.

I had hands-on experience with control problems in the automatic control course project last semester in which I built a cascade PID controller for a motor-driven cart.

Difference between open and closed loop?

an open-loop system never checks if the output matched the desired one, meaning that no feedback is returned to adjust the input. Example: the microwave runs for a certain time while not measuring the food temperature in real time.

A closed-loop system measures the actual output and uses it to adjust the input continually, meaning that feedback is fed to adjust the input and reach the desired output. Example: my own automatic control project which measures the position and velocity of the cart and adjusts the motor output accordingly.

What the feedback means?

It carries the actual output back to the controller so it can compute the error and adjust the input, so the system can self-correct itself.

Imagine a situation

The car starts braking because a sensor detects an obstacle 3 meters ahead. While braking is happening, the sensor keeps re-measuring the remaining distance.
At some point, the controller checks: "Based on the distance I've covered, am I going to stop in time?" If the feedback shows braking isn't happening fast enough, meaning it has covered more distance than it should have, the controller brakes harder so it can stop in time.

2- PID Control:

The PID controller computes a control value based on the error between the desired output and the actual one, using 3 terms each reacting to an aspect of that error.

Proportional (P): reacts to the current or the present error.

Integral (I): Reacts to the accumulated error over time by adding how much and how long the system has been off the target.

Derivative (D): reacts to the rate of change of the error, predicting how the error is heading (up or down) damping against fast changes.

The effects:

each term has a certain effect on the system

P: increases speed toward the target, but higher P risks overshoot, and even at normal values, it forces steady-state error.

I: eliminates steady-state error over time, but too much can cause overshoot and oscillation.

D: dampens oscillation and overshoot, smooths the response, but makes the noise bigger.

Tuning the controller:

To be honest, the tuning part was the hardest part in the whole project I've done in the course previously.

1- **Trial and error method:** I used this one in the project; it works by adjusting the gain by hand and observing the response of the system each time, until you reach the optimal one.

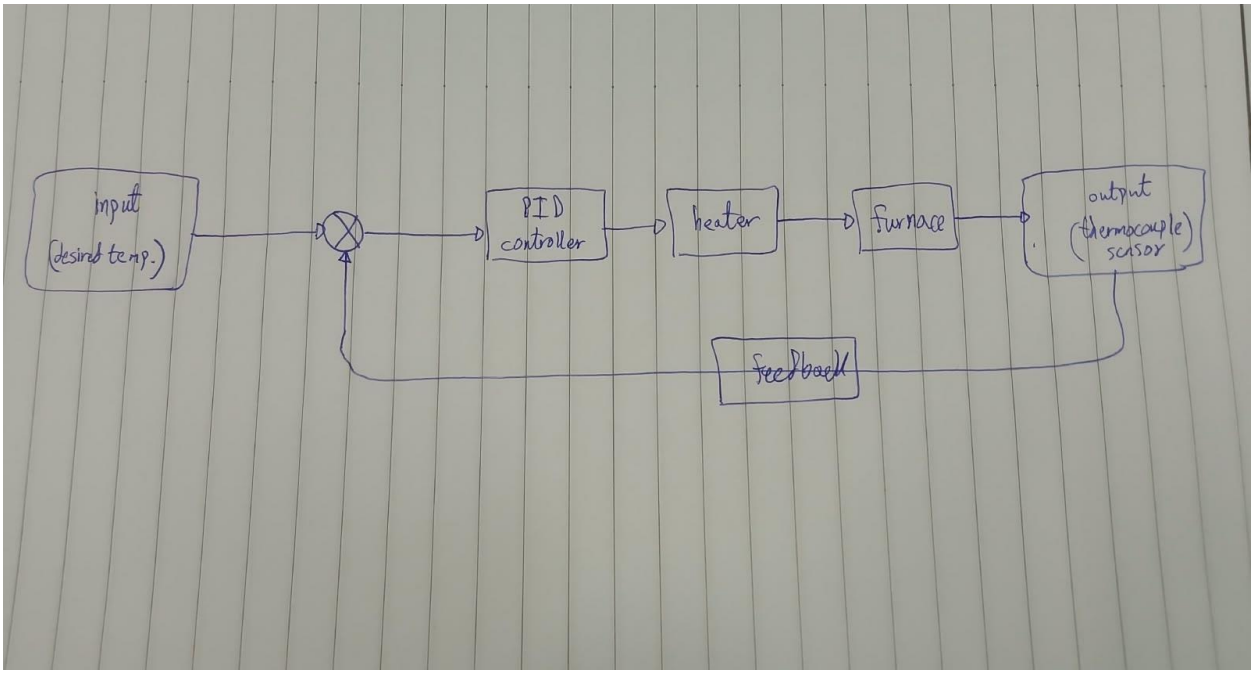
2- **Ziegler-Nichols method:** a classic approach, a step-by-step process, increase P until the system starts to oscillate, then use the ratios and formulas to calculate the I and D for the system to be optimal.

Imagine a situation

here we have a desired temperature for the furnace we want to reach, so we use a thermocouple to measure the actual temperature and give it to the PID controller through feedback, which adjusts it depending on the error for the furnace to raise or decrease the heat.

now the PID usage, if the furnace is below the target temperature, we give it large P to apply a lot of heat and then adjusting the D and I with the classic Ziegler-Nichols method, until we reach the desired output. The furnace needs D to prevent overshooting past the target before settling and needs I to eliminate the steady-state error when settling at a temperature.

BLOCK DIAGRAM



Limitations of the PID controllers:

PID assumes a roughly consistent behavior through the operation, but there are non-linear systems that can cause problems with that.

Changing dynamics over time, like losing weight when losing fuel in the car, could cause problems with the PID control method.

PID controls a single variable, but many systems have more than one changing parameter that needs to be controlled.

Alternative controller types:

When controlling a robotic arm's position, a robot arm joint being driven by a motor, where the arm needs to move through a wide range of angles. The effect of a given motor torque on the arm's actual motion isn't constant across that range. Adaptive controller fits better here because it continuously adjusts its gains based on the system's current behavior